山东大学

软件工程

Lumina 博客网站项目可行性分析报告

学院 计算机科学与技术学院

专业 计算机科学与技术

成员 叶舒涵 陈晓 陈奕妃 但功霜 张杨

2026 年 3 月 20 日

写在最前面：

本可行性报告分工：

成员	完成内容
叶舒涵	按照给出的目录补充其余内容、整理文档并上传平台
陈晓	项目概述-风险分析-结论
陈奕妃	项目背景与需求分析
但功霜	技术可行性相关部分
张扬	经济可行性相关部分

小组项目地址：https://github.com/Iridiums231549/Lumina

小组协作平台地址： 项目管理 - PingCode

小组成员账号：

成员	账号
叶舒涵	https://github.com/Iridiums231549
陈晓	https://github.com/onk016
陈奕妃	https://github.com/Liuyifan99
但功霜	https://github.com/1dan-gong
张杨	https://github.com/whatfk111

目录

1 引言.....9

1.1 标识.....9

1.2 背景.....9

1.3 项目概述.....10

1.4 文档概述.....10

2 引用文件.....11

3 可行性分析的前提.....11

3.1 项目的要求.....11

3.1.1 功能要求.....11

3.1.2 性能要求.....12

3.1.3 技术要求.....12

3.1.4 时间要求.....12

3.1.5 成本要求.....12

3.1.6 协作要求.....13

3.2 项目的目标.....13

3.3 项目的环境、条件、假定和限制.....13

3.3.1 环境.....13

3.3.2 条件.....14

3.3.3 假定.....14

3.3.4 限制.....14

3.4 进行可行性分析的方法.....14

4 可选的方案.....15

4.1 原有方案的优缺点、局限性及存在的问题.....15

4.2 可重用的系统，与要求之间的差距.....16

4.3 可选择的项目方案 1.....16

4.4 可选择的项目方案 2.....17

4.5 选择最终方案的准则.....17

5 所建议的项目.....18

5.1 对所建议的项目的说明.....18

5.2 数据流程和处理流程.....20

5.2.1 数据流程.....20

5.2.2 处理流程.....21

5.3 与原项目的比较.....21

5.4 影响.....22

5.4.1 设备.....22

5.4.2 软件.....22

5.4.3 运行.....22

5.4.4 开发.....22

5.4.5 环境.....23

5.4.6 经费.....23

5.5 局限性.....23

6 经济可行性(成本----效益分析).....24

6.1 投资.....24

6.1.1 基本建设投资.....25

6.1.2 其他一次性和非一次性投资.....25

6.2 预期的经济效益.....26

6.2.1 一次性收益.....26

6.2.2 非一次性收益.....26

6.2.3 不可定量的收益 26

6.2.4 收益/投资比 27

6.2.5 投资回收周期 27

6.3 运营维护成本 27

6.4 核心无形成本 28

6.5 成本控制办法 28

6.6 市场预测 29

6.7 经济可行性最终结论 29

7 技术可行性 30

7.1 现有资源评估 30

7.2 资源缺口及补救措施 31

7.3 技术风险评价 31

7.4 开发工具：Visual Studio Code 32

7.5 后端语言：C++ 33

7.6 数据库：MySQL 33

7.7 CASE 工具：MySQL Workbench， PingCode 等 34

7.8 技术架构的可行性总结.....35

7.9 存在的挑战与应对措施.....36

7.10 整体风险评估.....36

7.11 技术可行性结论.....37

8 法律可行性.....38

9 用户使用可行性.....39

9.1 行政管理和工作制度适配性.....39

9.2 使用人员的素质和培训要求.....39

9.3 用户使用可行性结论.....40

10 其他与项目有关的问题.....40

11 注解.....41

11.1 一般信息.....41

11.2 术语和定义.....41

11.3 缩略语表（按字母顺序排列）.....42

12 软件开发过程模型与敏捷开发对比分析.....42

12.1 传统软件开发过程模型与敏捷开发对比.....42

12.2 优缺点分析 43

12.3 在本博客项目中的应用选择.....44

13 Scrum 开发方法应用 44

13.1 Scrum 角色与职责 44

13.2 Scrum 核心流程.....45

13.3 项目 Sprint 规划（5 周） 45

14 项目风险分级与应对预案.....46

14.1 风险分级标准.....46

14.2 风险清单与应对预案 46

14.3 风险管理流程.....47

附录.....48

附录 A：团队分工表.....48

附录 B：数据库表结构设计（SQL 脚本）（待完成）.....49

附录 C：技术栈相关教程链接.....49

1 引言

1.1 标识

项目名称：博客网站

项目编号：BlogSystem-01

版本号：V1.0

发行号：20260320

小组名称：Lumina 小组

成员：叶舒涵　陈晓　陈奕妃　但功霜　张杨

开发平台：GitHub（仓库地址 https://github.com/Iridiums231549/Lumina）

协作平台：PingCode（网址：）

1.2 背景

随着互联网技术的飞速发展，博客作为一种重要的个人媒体和信息分享平台，在知识传播、技术交流、个人展示等方面发挥着不可替代的作用。从早期的个人独立博客到如今的轻量级写作平台，博客系统始终是互联网用户记录思想、分享经验、建立个人影响力的核心工具之一。

当前，虽然已有众多功能、风格各异的博客平台，但是开源系统部分功能过于复杂，超出本项目核心需求，部分系统功能简单，缺少管理员后台管理、评论

互动等必要功能。同时，对于学习软件工程的学生而言，从零构建一个简洁、可定制的博客系统，有助于深入理解 Web 开发的全流程，掌握前后端交互、数据库设计、用户认证、权限控制等核心技术。

本项目的目标是开发一个轻量级、功能清晰的博客网站，支持用户注册登录、文章管理、评论互动以及管理员后台管理等功能。系统旨在为用户提供一个纯净、易用的写作与交流环境，同时作为课程实践项目，检验和提升团队成员的软件工程能力。

1.3 项目概述

本系统是一个基于 Web 的博客管理系统，用途为为个人用户提供内容分享（文章发布、编辑、删除）、互动交流的平台，同时支持管理员对用户、文章、评论的全流程管理。系统定位为轻量化、易操作的个人内容分享与互动平台，核心面向个人内容分享爱好者，同时支持管理员进行后台管理，适配 PC 端使用。

项目为全新开发项目，无历史开发、运行及维护记录；项目投资方无，需方为开发小组，用户为个人内容分享爱好者，开发方为 5 人协作小组，无专门支持机构；当前运行现场为小组各成员本地设备，计划运行现场可选择免费云服务器或本地局域网；其他有关文档包括：团队分工表、数据库表结构设计 SQL 脚本、技术栈相关教程链接、项目开发进度表（PingCode 甘特图）、CASE 工具使用说明。

1.4 文档概述

本报告用于对博客网站进行可行性分析，明确项目在技术、经济、法律、用户使用等方面的可行性，为项目开发决策提供依据，同时指导项目后续开发流程的推进。文档内容涵盖引言、引用文件、可行性分析前提、可选方案、所建议系统、经济可行性、技术可行性、法律可行性、用户使用可行性等核心章节，全面覆盖项目可行性相关的各类要素。本文档无保密性和私密性要求，可供开发小组全体成员查阅、使用和修改，用于项目协作与进度推进。

2 引用文件

- 软件工程课程教材
- 可行性分析报告模板（FAR）
- 实验指导书

3 可行性分析的前提

3.1 项目的要求

3.1.1 功能要求

实现用户注册、登录、个人信息修改、密码重置，支持邮箱 / 用户名注册、密码加密存储与信息合法性验证，通过 Session/JWT 维护登录状态并支持注销；

实现文章发布、编辑、删除、浏览，支持 Markdown 文本编辑、文章分页展示，文章包含标题、正文、发布/修改时间等属性；

实现评论发布、删除，登录用户可发表评论，作者可管理自身文章评论，管理

员可审核、删除全站评论，评论按时间倒序展示；

实现管理员对用户、文章、评论的全流程管理，支持管理员查看/修改/删除用户信息、删除/隐藏违规文章、监控管理全站评论，可选实现博客标题、公告等全局系统设置；

界面精美、布局清晰、交互流畅，适配 PC 端。

3.1.2 性能要求

系统支持至少 100 个用户同时在线访问，文章列表页加载时间控制在 2 秒以内，数据库查询进行优化，避免 N+1 查询问题。

3.1.3 技术要求

基于 C++ 后端开发，搭配 MySQL 8.0 数据库，前端采用 HTML 开发，使用适配的 CASE 工具提升开发效率；代码符合分层架构要求，模块间低耦合，遵循统一代码规范；用户密码采用哈希算法加密存储，防止 SQL 注入、XSS 攻击、CSRF 攻击等常见 Web 漏洞，接口进行权限验证，确保系统稳定、数据安全，无明显 bug。

3.1.4 时间要求

开发周期控制在 5 周内，完成需求确认、设计、开发、联调、测试与交付。

3.1.5 成本要求

无额外经费投入，所有开发工具、软件均采用免费开源版本，人力成本为小组

成员课余 / 空闲时间投入。

3.1.6 协作要求

5 人分工明确，通过 Git、即时沟通工具及 CASE 工具实现高效协作，确保进度同步、代码无冲突。

3.2 项目的目标

- 核心目标：开发一款功能完整、操作流畅、界面精美的简易博客网站，满足个人用户内容分享与互动需求，完成课程设计/练手目标。
- 技术目标：提升团队成员 C++后端、前端、数据库设计、CASE 工具应用及项目协作能力，积累小型项目开发经验。
- 质量目标：确保系统稳定运行，核心功能无 bug，数据存储安全，界面适配 PC 端，交互流畅，用户体验良好。
- 进度目标：在 5 周内完成项目全流程开发，按时交付，确保各阶段任务有序推进、无严重延期。

3.3 项目的环境、条件、假定和限制

3.3.1 环境

开发环境：Windows 10 及以上操作系统，开发工具包括 VS Code（C++后端）、MySQL 8.0（数据库）、MySQL Workbench（数据库设计）、Github（代码管理）、PingCode（进度管理）、Apache JMeter（测试）及前端开

发工具；运行环境：PC端Windows操作系统，浏览器（Chrome、Edge等）。

3.3.2 条件

团队成员具备基础的 C++、前端、数据库设计及办公软件操作能力；所有开发工具、软件均可免费获取并安装；小组内沟通顺畅，可通过即时沟通工具同步进度、解决问题；具备稳定的网络环境，便于获取技术教程、同步代码。

3.3.3 假定

假定团队成员能够按时完成各自分工任务，无严重拖延；假定开发过程中无重大技术难题无法解决，或可通过查阅教程、交流沟通快速解决；假定所选用的免费开源工具能够正常运行，无兼容性问题；假定项目需求无重大变更，仅存在少量细节调整。

3.3.4 限制

人员限制：仅 5 人分工，无专业测试、运维人员，测试工作需全员参与；

技术限制：团队成员技术水平有限，不开发复杂功能；

时间限制：开发周期短，无法进行过度优化与功能扩展；

成本限制：无额外经费投入，无法使用付费工具、软件及云服务器；

功能限制：仅实现博客核心功能，不涉及高级功能。

3.4 进行可行性分析的方法

本次可行性分析采用以下方法，确保分析全面、准确、贴合项目实际：

- 文献调研法：查阅软件工程、C++后端开发、MySQL 数据库、CASE 工具应用相关文献及教程，确认技术可行性；
- 工具调研法：调研主流 CASE 工具的用途、技术特点，结合项目需求筛选适配工具；
- 成本效益分析法：核算项目人力、时间、软件、服务器等成本，分析项目收益，评估经济可行性；
- 风险评估法：识别项目开发过程中可能存在的技术、协作、需求、工具等风险，分析风险影响程度及应对措施；
- 分工适配法：结合团队 5 人分工，分析各角色任务与技术能力的匹配度，评估操作与协作可行性；
- 经验参考法：参考同类小型博客项目的开发案例，结合团队实际情况，判断项目可行性。

4 可选的方案

4.1 原有方案的优缺点、局限性及存在的问题

本次项目为全新开发，无原有方案，故本条款不适用。若参考同类小型博客项目的常规开发方案，其优缺点、局限性如下：

- 优点：功能成熟、技术路线清晰，可参考案例多，开发难度低；
- 缺点：多采用 PHP/Java 后端技术栈，与本团队 C++后端技术路线不匹配；部分方案功能复杂，不适合小型团队短周期开发；

- 局限性：未结合本团队 5 人分工特点，未适配免费开源 CASE 工具，协作效率较低；
- 存在的问题：技术栈适配性差，开发周期难以控制，不适合本项目的时间与人员限制。

4.2 可重用的系统，与要求之间的差距

目前开源社区存在多款简易博客网页，可作为可重用项目参考，其与本项目要求的差距如下：

- 技术栈差距：开源可重用系统多采用 PHP+MySQL 或 Java+MySQL 技术栈，与本项目 C++后端技术路线不匹配，无法直接重用后端代码；
- 功能差距：开源项目部分功能过于复杂，超出本项目核心需求；部分项目功能简单，缺少管理员后台管理、评论互动等必要功能；
- 协作差距：开源项目未结合本团队 5 人分工设计开发流程，无法适配团队协作模式，且未集成适配的 CASE 工具，不利于进度管理与代码控制；

4.3 可选择的项目方案 1

方案名称：C++后端+MySQL 数据库+基础前端+核心 CASE 工具方案（本次选定方案）

核心内容：后端采用 C++开发，实现核心接口；数据库采用 MySQL 8.0，使用 MySQL Workbench 设计表结构；前端采用 HTML 简易 UI 组件库，实现页面开发与交互；选用 Git（代码管理）、PingCode（进度管理）、Apache

JMeter（测试）3 款 CASE 工具，辅助项目开发；5 人分工明确，按“需求确认-设计-并行开发-联调-测试-交付”流程推进。

优点：技术栈与团队成员能力匹配，CASE 工具适配项目需求，可提升开发效率；功能贴合项目核心需求，开发周期短，成本低；分工清晰，协作顺畅，风险可控。

缺点：技术栈相对小众，C++后端对接 MySQL 的参考案例较少；无专业测试人员，测试覆盖度有限；功能较为基础，后续扩展空间需进一步挖掘。

4.4 可选择的项目方案 2

方案名称：Java 后端+MySQL 数据库+前端+简化 CASE 工具方案

核心内容：后端采用 Java 技术栈（Spring Boot）开发，数据库采用 MySQL 8.0，前端与方案 1 一致；仅选用 Git（代码管理）一款 CASE 工具，不使用建模、进度管理、测试类 CASE 工具；开发流程简化，减少设计与测试环节，优先完成核心功能开发。

优点：Java 后端参考案例多，技术成熟，接口开发难度低；开发流程简化，可缩短开发周期；

缺点：与团队 C++后端技术路线不匹配，团队成员需快速学习 Java 技术，学习成本高；缺少建模、进度管理、测试类 CASE 工具，开发流程不规范，协作效率低，bug 排查难度大；功能质量难以保障，后期维护困难。

4.5 选择最终方案的准则

结合本项目的人员分工、技术能力、时间限制、成本要求及核心需求，确定最终方案的选择准则如下，据此选定**方案 1** 作为最终开发方案：

- 技术适配准则：方案需与团队成员技术能力匹配，优先选用 C++后端技术栈，降低学习成本；
- 效率提升准则：方案需集成适配的 CASE 工具，覆盖建模、代码管理、进度管理、测试全流程，提升开发与协作效率；
- 成本可控准则：方案所需工具、软件均为免费开源，无额外经费投入，符合成本要求；
- 进度适配准则：方案开发流程清晰，可在 5 周内完成开发，符合时间限制；
- 功能匹配准则：方案可实现项目核心需求，功能完整、无冗余，适配小型博客网站定位；
- 风险可控准则：方案潜在风险少，可通过合理措施应对，确保项目顺利推进、按时交付。

5 所建议的项目

5.1 对所建议的项目的说明

所建议的系统为简易博客网站，采用方案 1（C++ 后端 + MySQL 数据库 + 基础前端 + 核心 CASE 工具）开发，定位为轻量化、易操作的个人内容分享与互动平台，适配 PC 端使用，核心面向个人用户，同时支持管理员进行后台管理，为用户提供纯净、易用的写作与交流环境，同时满足团队课程实践的学习与能

力提升目标。

系统整体架构分为前端层、后端层、数据层三层，各层协同工作，确保系统稳定运行，模块间低耦合，符合可扩展性与可维护性要求。

- ✓ 前端层：负责页面展示与用户交互，采用 HTML+CSS+JavaScript + 简易 UI 组件库开发，实现用户注册登录、文章浏览与 Markdown 编辑、评论互动、个人信息修改及管理员后台操作等页面，支持文章分页、评论倒序展示，界面简洁清晰，操作流程符合用户直觉；
- ✓ 后端层：负责业务逻辑处理与接口开发，采用 C+++MySQL Connector/C++，基于 cpp-httplib 库搭建 HTTP 服务，使用 jsoncpp 库处理前后端 JSON 格式数据交互，实现用户认证、文章管理、评论管理、权限控制等核心接口，对接前端与数据库，完成身份验证、权限校验、数据处理等核心业务，实现防 SQL 注入、XSS 攻击等安全防护；
- ✓ 数据层：负责数据存储与管理，采用 MySQL 8.0 数据库，通过 MySQL Workbench 设计表结构并绘制 E-R 图，设计用户、文章、评论、角色等数据表，建立合理的关联关系，确保数据安全、查询高效，同时预留标签、分类等功能的扩展空间。

系统集成 4 款 CASE 工具，分别用于建模（Microsoft Visio）、代码管理（Git）、进度管理（PingCode）、接口测试（Apache JMeter），覆盖项目全流程开发，提升开发效率与协作质量，确保项目按时交付。

系统支持**普通用户、管理员、游客**三类用户角色，不同角色拥有不同操作权限：游客可浏览公开文章和评论；普通用户注册登录后可管理个人文章、发表评论；

管理员拥有全平台管理权限，可管理所有用户、文章与评论。

5.2 数据流程和处理流程

5.2.1 数据流程

系统数据主要包括用户数据、文章数据、评论数据三大类，数据流程如下：

1. 用户数据流程：用户注册→前端提交注册信息（用户名、密码、邮箱等）→后端接收并验证信息→验证通过后将数据存入 MySQL 数据库→用户登录时，前端提交登录信息→后端查询数据库验证信息→验证通过后返回登录成功结果，授予对应权限（普通用户/管理员）。

2. 文章数据流程：普通用户发布文章→前端提交文章信息（标题、内容、标签、封面图等）→后端接收并验证信息→验证通过后将数据存入数据库→用户浏览文章时，前端请求文章数据→后端查询数据库并返回数据→前端渲染展示文章；用户编辑/删除文章时，前端提交操作请求→后端验证权限后，更新/删除数据库中的文章数据。

3. 评论数据流程：用户发表评论→前端提交评论信息（文章 ID、评论内容、用户 ID 等）→后端接收并验证信息→验证通过后将数据存入数据库→用户浏览文章时，后端同步查询该文章的评论数据并返回→前端渲染展示评论；用户删除自己的评论时，前端提交删除请求→后端验证权限后，删除数据库中的评论数据。

4. 管理员数据流程：管理员登录后，请求用户/文章/评论数据→后端查询数据库并返回全部数据→管理员执行增删改查操作→后端验证管理员权限后，更

新/删除数据库中的对应数据。

5.2.2 处理流程

系统核心处理流程如下，整体遵循“前端请求-后端处理-数据交互-前端渲染”的逻辑：

1. 初始化流程：启动系统→后端初始化 MySQL 数据库连接→前端加载页面→完成系统初始化，等待用户操作。

2. 用户操作处理流程：用户在前端执行操作（注册、登录、发布文章、评论等）→前端封装请求参数→发送请求至后端接口→后端接收请求，验证参数合法性与用户权限→调用对应业务逻辑处理→与数据库进行数据交互（查询、新增、更新、删除）→后端封装处理结果，返回至前端→前端接收结果，渲染页面，反馈给用户。

3. 管理员操作处理流程：管理员登录→前端请求管理员权限验证→后端验证通过后，返回管理员后台页面→管理员执行管理操作（用户/文章/评论管理）→前端提交操作请求→后端验证权限，处理业务逻辑，与数据库交互→返回处理结果，前端渲染展示。

4. 系统关闭流程：用户退出登录→前端发送退出请求→后端清除用户登录状态→关闭数据库连接→系统正常关闭。

5.3 与原项目的比较

本项目为全新开发项目，无原系统，故本条款不适用。

5.4 影响

5.4.1 设备

开发设备要求：小组各成员需配备一台安装 Windows 10 及以上操作系统的 PC，内存不低于 8GB，硬盘空间不低于 50GB，确保能够正常安装开发工具、数据库及 CASE 工具；运行设备要求：用户使用 PC 端（Windows 操作系统），浏览器版本为 Chrome 90.0 及以上、Edge 90.0 及以上，确保页面正常渲染与交互。

5.4.2 软件

开发软件要求：后端开发需安装 VS Code、MySQL 8.0、MySQL Connector/C++；数据库设计需安装 MySQL Workbench；前端开发需安装 HTML/CSS/JavaScript 编辑工具（可使用 VS Code）、简易 UI 组件库；CASE 工具需安装 Microsoft Visio、Git、PingCode、Apache JMeter；所有软件均采用免费开源版本，确保兼容性与可用性。

5.4.3 运行

运行环境要求：PC 端浏览器需支持 HTML5、CSS3、JavaScript，确保页面交互流畅；若线上部署，云服务器需开启 3306（MySQL 端口）、80（HTTP 端口），确保数据库与系统能够正常访问；系统运行过程中，需保证设备网络稳定，避免数据传输中断；数据库需定期备份，防止数据丢失。

5.4.4 开发

开发要求：遵循既定分工，后端人员负责接口开发与数据库对接，前端人员负责页面开发与交互，数据库负责人负责表结构设计与 ER 图绘制，美工负责界面美化，文档协调员负责进度管理与文档编写；使用 Git 进行代码管理，规范代码提交流程，避免代码冲突；使用 PingCode 跟踪开发进度，确保各阶段任务按时完成；使用 Apache JMeter 进行接口测试，及时排查 bug；开发过程中，及时同步进度、沟通问题，确保开发流程顺畅。

5.4.5 环境

开发环境要求：小组各成员开发环境需保持一致（软件版本、配置参数），避免兼容性问题；开发过程中，保持网络稳定，便于获取技术教程、同步代码与文档；开发环境需保持清洁，避免恶意软件、病毒影响开发进度与数据安全。运行环境要求：PC 端浏览器需定期更新，确保系统兼容性；云服务器（若有）需定期维护，检查运行状态，及时修复异常。

5.4.6 经费

本项目无额外经费投入，经费要求如下：开发工具、软件均采用免费开源版本，无需支付授权费用；人力成本为小组成员课余/空闲时间投入，无需支付薪酬；若线上部署，可选用免费云服务器，无需支付服务器费用；无其他经费支出，确保成本可控。

5.5 局限性

所建议的系统存在以下局限性，需在开发过程中重点关注，后续可逐步优化：

- ✓ 功能局限性：仅实现博客核心功能（用户管理、文章管理、评论管理），不支持高级功能（如用户关注、文章点赞、分类管理、移动端适配等），功能较为基础；
- ✓ 技术局限性：采用 C++后端技术栈，对接 MySQL 的参考案例较少，部分复杂接口开发难度较高；无专业测试人员，测试覆盖度有限，可能存在潜在 bug；
- ✓ 性能局限性：系统仅适配小型用户群体，不支持高并发访问，若用户量增加，可能出现响应缓慢的问题；数据库未进行深度优化，大数据量查询效率可能较低；
- ✓ 扩展性局限性：系统架构设计较为简单，后续扩展功能时，可能需要修改核心代码，维护成本较高；
- ✓ 安全局限性：仅实现基础的数据加密与权限控制，未采用高级安全防护措施（如防 SQL 注入、防 XSS 攻击等），存在一定的安全风险。

6 经济可行性(成本----效益分析)

本博客网站为课程小组实践项目，无商业盈利目的，仅围绕课程学习、技术实践和团队协作开展开发，本次经济可行性分析从投资成本、运营维护成本、核心无形成本、成本控制办法和项目非商业收益等方面展开，分析项目在经济层面的可落地性，结论为项目全程无资金投入压力，经济层面完全可行。

6.1 投资

本项目为非商业项目（课程设计 / 练手项目），投资主要包括人力投资、时间投资，无额外经费投入，开发、运营、维护全流程均依托小组成员自有资源和免费工具完成，具体如下：

6.1.1 基本建设投资

无专门的基本建设投资，开发环境、设备均为小组成员自有 PC（笔记本 / 台式机），无需额外购买服务器、测试机等硬件设备；开发软件（VS Code、MySQL 社区版等）、CASE 工具均为开源免费版本，资料均通过网络免费获取，无资料购买、软件授权等费用。

6.1.2 其他一次性和非一次性投资

· 技术管理费：0 元，技术支持通过网络教程、开源社区、课程老师指导免费获取，无需支付额外费用；

· 培训费：0 元，团队成员通过自学、互相交流掌握开发工具与 CASE 工具的使用，无需参加付费培训；

· 管理费：0 元，无专门的管理费用支出；

· 人员工资、奖金：0 元，小组成员利用课余 / 空闲时间开发，无薪酬支出；

· 差旅费：0 元，项目开发采用线上协作模式，无需线下聚集，无差旅费支出；

· 协作与存储成本：0 元，使用 GitHub 免费版完成代码托管和团队协作，腾讯文档、WPS 免费版完成文档编写，均满足项目需求；

· 时间投资：项目开发利用课余时间完成，小组每人每周大概投入 3-5 小时，主要用于小组讨论、写代码、做测试、写文档，总工时约 70-150 小时，为小组成员的空闲时间投入，无直接经济成本。

项目总投资为 0 元，仅需投入小组成员的时间与精力，全程无任何资金支出。

6.2 预期的经济效益

本项目为非商业项目，无直接金钱经济效益，核心收益为非经济收益与潜在成果收益，这些收益远大于小组投入的时间和学习成本，具体如下：

6.2.1 一次性收益

完成课程设计 / 练手目标，获得对应课程实践成果与学分；开发完成一款可直接使用的简易博客网站，满足个人内容分享平台的基础需求；积累完整的小型项目开发案例与可复用的实践成果，项目开发的代码、设计文档、可行性报告可作为专业作品集，为后续学习、求职提供支撑。

6.2.2 非一次性收益

提升团队成员的技术能力，亲手掌握 C++ 后端开发、前端页面制作、数据库设计、CASE 工具应用、Git 协作等网站开发全流程技能，熟练运用软件工程必备工具，这些技能可直接迁移至后续课程学习和专业实践中，完成专业能力的重要积累；锻炼团队协作能力，以 5 人小组为单位完成从分工、沟通到问题解决的全流程项目开发，培养任务分配、高效沟通的团队意识，提升软件工程所需的协作能力；掌握小型项目的开发流程与可行性分析方法，为后续参与更复杂项目奠定基础。

6.2.3 不可定量的收益

团队成员之间的协作默契提升，培养团队合作意识；提升个人学习能力与自主

解决问题的能力；增强对软件工程、项目管理的理解与认知；为开源社区提供可参考的小型 C++博客项目案例。

6.2.4 收益/投资比

项目总投资为 0 元，收益主要为非经济效益与能力提升，无法用具体金额量化，故收益 / 投资比无法计算。但从学习与经验积累角度，项目带来的专业能力、协作能力提升及课程成果收益，远高于投入的时间与精力，具备极高的非经济价值。

6.2.5 投资回收周期

本项目无直接经济投资，故无投资回收周期。从学习与经验积累角度，项目开发过程中，团队成员即可逐步获得收益（技术提升、经验积累），开发完成后，可长期使用博客网站、参考项目案例，收益具有持续性。

6.3 运营维护成本

本项目是课程实践成果，不需要像商业网站一样长期上线运营，仅需完成开发后做短期的功能测试、课程演示和简单维护，这一阶段同样几乎无资金成本，具体如下：

- ✓ 部署展示：课堂演示网站功能可直接在本地电脑运行，或使用阿里云、腾讯云的学生免费云服务器 / 试用服务器，无需租赁正式服务器，部署成本为 0；
- ✓ 后期维护：网站的 bug 修复、功能微调、数据整理等工作，全部由小组成

员分工完成，无需聘请专业技术人员和运维人员，人力维护成本为 0；

- ✓ 域名与备案：仅做课程展示，无需公网访问，因此无需注册域名及进行 ICP 备案，这部分成本为 0；
- ✓ 数据存储：项目测试数据（模拟用户信息、文章、评论等）量小，直接存储于本地数据库，无需购买云存储、数据库备份服务，数据成本为 0。

6.4 核心无形成本

项目无实际资金支出，小组成员投入的时间成本和学习成本是最主要的 “经济投入”，且该部分成本可通过合理规划灵活把控，具体说明如下：

- ✓ 时间成本：投入的课余时间无硬性要求，可根据成员课表、空余时间灵活调整，如周末多投入、上课周少投入；同时小组按成员优势分工，避免重复工作，杜绝时间浪费，保障时间投入的效率；
- ✓ 学习成本：开发中遇到不熟悉的技术（开发框架、Git 协作流程、CASE 工具使用等）时，成员需投入时间学习，但学习资源均为免费，且可借助课程老师、助教的指导，无需付费报课；同时该学习为 “学用结合”，带着项目需求学习的内容可直接应用于开发，并非无用投入。

6.5 成本控制办法

为让小组的时间和学习投入更高效，避免无效做功，结合项目非商业、课程实践的特点，制定简单易执行的成本控制办法，核心为**不花钱、不浪费时间**，具体如下：

- ✓ 全程只用免费资源：所有开发、协作、展示、部署环节，仅选择开源免费的软件、平台和测试资源，坚决不使用任何付费工具、付费会员，从源头杜绝资金支出；
- ✓ 按能力精准分工：根据小组成员的技术优势分配工作，让专业的人做专业的事，避免因 “能力不匹配” 导致的重复工作和时间浪费，提升开发效率；
- ✓ 定期简短小组沟通：每周花半小时到一小时开展小组会议，同步开发进度、及时解决遇到的问题，避免因问题搁置导致后续返工，节省整体开发时间；
- ✓ 聚焦核心功能开发：仅开发课程要求的核心功能（用户注册登录、发表 / 管理文章、评论互动、管理员后台管理），不做复杂的界面美化、高级交互等非核心功能，降低开发难度，减少学习和开发时间投入。

6.6 市场预测

本项目为非商业项目，不涉及市场运营与盈利，故无专门的市场预测。从个人内容分享市场来看，简易博客网站仍有一定的需求，适合个人用户、学生群体用于内容记录与分享；从技术学习市场来看，小型 C++ 博客项目案例可满足学生、新手开发者的学习需求，具有一定的参考价值。若后续对系统进行功能扩展（如增加移动端适配、高级互动功能），可尝试面向小型群体提供免费使用服务，具备一定的市场潜力。

6.7 经济可行性最终结论

综合以上所有分析，本博客网站项目作为课程小组实践项目，全程无任何资金成本，开发、运营、维护的所有环节都能通过免费资源和小组成员自有设备完

成；唯一的投入是小组成员的课余时间和针对性学习成本，且这部分成本能通过合理分工、定期沟通、聚焦核心开发等办法灵活把控，不会造成过度投入，成本控制简单易执行。

同时，项目能为小组成员带来显著的技术能力、团队协作能力提升，助力顺利完成课程实验要求，积累可复用的专业实践成果，其带来的非商业收益远大于投入的无形成本。

综上，本项目在经济层面具备完全的可行性，无需任何资金投入，且投入的时间和学习成本能带来实实在在的能力提升与课程成果，完全符合课程实践项目的经济要求和学习目标。

7 技术可行性

本项目开发小组现有资源（人员、环境、设备、技术条件）基本能够满足项目实施要求，部分潜在缺口可通过合理措施补救，技术层面完全可行，具体分析如下：

7.1 现有资源评估

人员资源：5 人分工明确（前端 1 名、C++后端 2 名、美工设计 1 名、文档协调 1 名），均具备基础的对应领域技术能力，能够完成各自分工任务；虽无专业测试、运维人员，但可通过全员参与测试，弥补人员缺口。

环境资源：小组各成员均具备符合要求的开发设备（Windows PC），网络环境稳定，可正常获取技术教程、同步代码与文档；开发环境可快速搭建，所有软

件均可免费安装。

设备资源：开发设备、运行设备均为自有或免费获取，无需额外投入；若需线上部署，可选用免费云服务器，满足部署需求。

技术条件：团队成员熟悉 C++、HTML、MySQL 等核心技术，能够完成后端接口开发、前端页面开发、数据库设计；通过自学可快速掌握 CASE 工具的使用；有成熟的技术教程、参考案例可供查阅，能够解决开发过程中的常见问题。

7.2 资源缺口及补救措施

缺口 1：无专业测试人员，测试覆盖度有限，可能存在潜在 bug；补救措施：制定详细的测试计划，全员参与测试，重点测试核心功能；使用 Apache JMeter 进行接口测试，提升测试效率与覆盖度；参考同类项目测试案例，补充测试场景。

缺口 2：C++后端对接 MySQL 的参考案例较少，部分复杂接口开发难度较高；补救措施：提前查阅 MySQL Connector/C++官方文档，学习接口开发方法；分工协作，后端人员互相审核代码、解决技术难题；参考开源社区相关案例，简化复杂接口设计，优先实现核心功能。

缺口 3：团队成员对部分 CASE 工具（如 PingCode、Apache JMeter）的使用不够熟悉；补救措施：提前查找简易操作教程，组织全员简单学习；安排负责对应工具的成员分享操作技巧，及时解决操作问题；在开发过程中逐步熟练工具使用，提升操作效率。

7.3 技术风险评价

项目开发过程中可能存在的技术风险及评价如下，所有风险均为中低风险，可通过合理措施应对，不影响项目可行性：

风险 1：C++后端与 MySQL 对接异常，导致数据读写失败；风险等级：中；评价：可通过提前测试数据库连接、确认软件版本兼容、规范字段类型对应等措施，降低风险发生概率，即使发生，也可快速排查解决，不影响整体开发进度。

风险 2：前端与后端接口对接出现数据不一致，导致功能异常；风险等级：中；评价：可通过提前约定接口格式、同步字段说明、联调阶段密切配合等措施，避免风险发生，发生后可及时调整接口参数，快速解决。

风险 3：数据库设计不合理，导致后期维护困难；风险等级：低；评价：可通过数据库设计阶段与后端人员共同审核、遵循规范设计、添加必要约束与索引等措施，避免设计不合理问题，确保数据库可维护性。

风险 4：CASE 工具操作不熟练，影响开发效率；风险等级：低；评价：可通过提前学习、互相分享、实践操作等方式，快速掌握工具使用方法，逐步提升效率，对开发进度影响较小。

7.4 开发工具：Visual Studio Code

本系统拟采用 Visual Studio Code 作为主要开发工具。

VS Code 是微软推出的一款免费代码编辑器，团队成员在课程学习和上机实践中均有使用，对其操作较为熟悉。编写 C++ 代码时，安装官方的 C/C++ 插件后，可实现代码高亮、智能补全、调试等功能；编辑器内置终端，可直接编译

运行程序，无需来回切换窗口，提升开发效率。

VS Code 可通过安装插件连接 MySQL 数据库，方便开发过程中查看数据表结构；同时集成了 Git 工具，便于小组协作开发时的代码版本管理。综合来看，使用 VS Code 进行本系统的开发在技术上具备可行性。

7.5 后端语言：C++

本系统拟采用 C++ 作为后端开发语言。

选择 C++ 主要基于以下考量：一是 C++ 为团队成员专业主修语言，成员对其语法相对熟悉，开发过程中遇到问题时更易解决，降低开发试错成本；二是 C++ 运行效率高，对于博客网站需处理多个用户并发请求的场景，在性能上具备一定优势；三是 C++ 拥有丰富的第三方库，可借助 cpp-httplib 库快速搭建 HTTP 服务，使用 jsoncpp 库处理前后端交互的数据格式，无需从头实现所有功能，提升开发效率。

当然，使用 C++ 开发网站也存在一定挑战，相较于 PHP、Python 等专为 Web 开发设计的语言，C++ 处理 HTTP 请求更偏向底层，需要编写更多代码。但团队通过查阅相关资料，找到了 xblog、xblogmini 等使用 C++ 开发博客系统的成功案例，证明该技术路线具备实操性。因此，使用 C++ 实现本系统的后端功能在技术上是可行的。

7.6 数据库：MySQL

本系统拟采用 MySQL 作为数据库管理系统。

MySQL 是目前广泛使用的开源关系型数据库，我们在数据库课程中学过它的基本操作。MySQL 提供了 C API，可以直接在 C++程序中进行数据库的连接、查询、插入、更新和删除操作。本系统需要存储用户信息、文章内容、评论数据等，这些数据之间有明确的关联关系（如一个用户有多篇文章，一篇文章有多条评论），用关系型数据库来管理比较合适。

根据我们目前所学的知识，可以通过以下 MySQL C API 函数实现基本的数据库操作：

功能	所用函数	说明
连接数据库	mysql_init(), mysql_real_connect()	初始化并连接到 MySQL 服务器
执行 SQL	mysql_query()	执行增删改查语句
查询数据	mysql_store_result(), mysql_fetch_row()	获取并遍历查询结果
关闭连接	mysql_close()	释放数据库连接资源

综合来看，使用 MySQL 作为本系统的数据库在技术上是可行的。

7.7 CASE 工具：MySQL Workbench， PingCode 等

在数据库设计阶段，我们计划使用 MySQL Workbench 来辅助设计。

MySQL Workbench 是 MySQL 官方提供的图形化管理工具，可以用可视化的方式设计数据库表结构，画 E-R 图，还能自动生成建表语句。

用这些工具可以帮助我们更好地设计用户表、文章表、评论表之间的关系，比

如一个用户对应多篇文章、一篇文章对应多条评论这种一对多的关系，画成图之后看起来更清楚，也方便小组讨论和修改。

7.8 技术架构的可行性总结

本系统拟采用的技术架构如下：

- 前端：HTML/CSS/JavaScript 页面
- 后端：C++处理业务逻辑
- 数据库：MySQ 存储数据
- 数据交互：JSON 格式
- HTTP 服务：使用 cpp-httplib 库

针对报告中提出的三个主要模块，初步的技术实现思路如下：

用户注册登录模块：前端提交用户名密码，后端 C++程序接收后，通过 MySQL C API 查询数据库，验证用户身份。验证通过后返回登录成功信息，前端跳转到对应用户的个人博客页面。

管理员后台管理模块：管理员登录后，可以发送请求到后端，后端执行相应的 SQL 语句（如查询所有用户、删除违规文章、修改用户信息等），并将操作结果返回前端。

用户文章管理及评论模块：用户可以发表、修改、删除自己的文章，后端根据请求执行相应的数据库操作。浏览文章时可以查看评论、发表评论，这些功能同样通过后端的数据库操作来实现。

7.9 存在的挑战与应对措施

虽然从技术角度分析，这个方案是可行的，但在实际开发过程中可能会遇到一些问题：

- **C++处理 HTTP 请求相对复杂**：相比于专门的 Web 开发语言，C++处理网络请求需要写更多底层代码。应对措施是使用 cpp-httplib 这样的轻量级库，封装好常见的 HTTP 操作，降低开发难度。
- **数据库操作的代码量较大**：每次数据库操作都需要写连接、执行、获取结果、释放资源等重复代码。应对措施是封装一个数据库操作类，把常用的增删改查写成函数，避免重复造轮子。

7.10 整体风险评估

任何技术方案都存在一定的风险，我们对本系统可能遇到的风险进行了分析，并初步想了些应对办法。

风险类型	风险描述	可能性	影响程度	应对措施
技术经验不足	小组成员之前没有用 C++做过完整的 Web 项目，对 HTTP 协议、网络编程这些不太熟悉，开发过程中可能会卡住。	中	高	提前安排时间学习相关知识，多看一些开源项目的代码，遇到问题及时问老师或者上网查资料。
第三方库	cpp-httplib、jsoncpp 这	低	中	统一固定库的版本，不要追

兼容性问题	些库可能会有版本更新，或者在不同操作系统上编译时报错。			新；在小组内统一开发环境，都用 Windows 系统，减少环境差异。
数据库设计不合理	如果一开始表结构设计得不好，比如字段类型选错、外键关系混乱，后期改起来会很麻烦。	中	高	先用 CASE 工具（比如 MySQL Workbench）画 E-R 图，小组一起讨论确认后再生成建表语句，尽量减少设计上的问题。
开发进度延误	大家都是利用课余时间做项目，可能会有考试、实习或者其他事情冲突，导致进度跟不上。	中	高	先用 CASE 工具（比如 MySQL Workbench）画 E-R 图，小组一起讨论确认后再生成建表语句，尽量减少设计上的问题。
安全性问题	比如用户密码明文存储、SQL 注入攻击等，可能会导致用户数据泄露。	低	高	密码要用哈希算法加密后再存数据库，执行 SQL 语句时对用户输入的内容做检查，避免拼接字符串。

7.11 技术可行性结论

综合现有资源评估、资源缺口补救措施、技术风险评价及各技术环节的可行性分析，本项目现有资源基本满足实施要求，潜在缺口可通过合理措施弥补，技术风险整体可控，开发难度与团队成员技术能力相匹配。同时，选定的开发工

具、后端语言、数据库及 CASE 工具均适配项目需求，技术架构清晰，模块实现思路明确，针对开发中可能遇到的挑战也制定了具体应对方案。综上，本项目在技术层面完全可行。

8 法律可行性

本项目开发过程中，严格遵守相关法律法规，无侵权、违法及相关责任风险，法律层面完全可行，具体分析如下：

- 软件授权风险：项目所使用的开发工具（VS Code、MySQL、MySQL Workbench）、CASE 工具（Microsoft Visio、Git、PingCode、Apache JMeter）及前端组件库，均采用免费开源版本，符合软件授权协议，无软件侵权风险；
- 内容侵权风险：系统仅提供内容发布、分享功能，用户发布的文章、评论等内容由用户自行负责，开发小组不参与内容创作，无内容侵权责任；可在系统中添加内容规范提示，引导用户发布合法合规内容，降低间接风险；
- 数据安全法律风险：系统收集的用户数据（用户名、密码、邮箱等）仅用于用户认证与系统运行，严格遵守《网络安全法》《个人信息保护法》，不泄露、不滥用用户数据，对用户密码进行加密存储，确保数据安全，无违法风险；
- 其他法律风险：项目为非商业项目，不涉及盈利、广告、版权交易等活动，无其他违法违规行为；开发过程中不抄袭、不盗用他人代码，确保项目代码的原创性，无侵权风险。

综上，本项目开发与运行均符合相关法律法规，无侵权、违法及责任风险，法律可行性极高。

9 用户使用可行性

本项目开发的简易博客网站，用户使用门槛低，适配用户使用习惯，结合用户单位（课程设计，用户为学校；练手项目，用户为个人）的行政管理、工作制度及使用人员素质，用户使用可行性极高，具体分析如下：

9.1 行政管理和工作制度适配性

本项目为课程设计项目，用户单位为学校，系统开发与使用符合学校的课程管理、教学工作制度，可作为课程实践成果提交，适配学校的行政管理要求；

若为练手项目，用户为个人，无严格的行政管理与工作制度限制，系统使用灵活，可根据个人需求调整使用方式，适配个人使用习惯。

9.2 使用人员的素质和培训要求

- 使用人员素质：系统面向普通用户与管理员，普通用户只需具备基础的电脑操作能力、浏览器使用能力，即可完成注册、登录、发布文章、评论等操作；管理员只需具备基础的电脑操作能力，即可完成用户、文章、评论的管理操作，使用人员素质要求低，适配大多数用户。
- 培训要求：系统界面简洁、操作流程清晰，普通用户无需专门培训，通过页面提示即可快速上手；管理员操作较为简单，可通过简单的口头指导或文档说明，快速掌握管理功能的使用，无需进行复杂培训。若有新增用户，可提供

简易使用手册，进一步降低使用难度。

9.3 用户使用可行性结论

系统使用门槛低、操作简单，适配使用人员的素质要求，无需复杂培训，且符合用户单位的行政管理与工作制度，用户使用可行性极高，能够满足普通用户与管理员的使用需求。

10 其他与项目有关的问题

本项目未来可能的变化及应对措施如下，提前预判变化，确保项目具有一定的灵活性与可扩展性：

- 需求变化：未来可能新增高级功能（如用户关注、文章点赞、分类管理、移动端适配等）；应对措施：项目开发过程中，遵循“模块化设计”原则，预留功能扩展接口，后续新增功能时，可减少核心代码修改，降低维护成本；
- 技术变化：未来 C++后端、MySQL 数据库、CASE 工具可能出现新版本，存在兼容性问题；应对措施：开发过程中，选用稳定的软件版本，避免使用最新测试版；后续若需升级软件，提前测试兼容性，逐步升级，确保系统稳定运行；
- 用户需求变化：未来用户可能对界面风格、操作流程提出新的要求；应对措施：系统界面设计采用可修改的模板，操作流程简洁灵活，后续可根据用户反馈，快速调整界面与操作流程；
- 团队变化：未来可能出现团队成员部分缺席，影响开发进度；应对措施：

完善项目文档，规范代码编写，确保其他成员能够快速接手工作；每周同步开发进度，及时整理项目资料，降低人员变动带来的影响。

11 注解

11.1 一般信息

本文档为博客网站项目的可行性分析报告，基于软件工程相关原理，结合项目实际情况，从技术、经济、法律、用户使用等多个维度，全面分析项目可行性，为项目开发决策提供依据。文档的编写遵循“客观、全面、准确、贴合实际”的原则，所有分析均基于团队现有资源、技术能力及项目需求，确保分析结果具有参考价值与可操作性。

11.2 术语和定义

- CASE 工具：Computer-Aided Software Engineering，即计算机辅助软件工程工具，用于辅助软件开发生命周期的各个阶段，提升开发效率、规范开发流程；
- C++后端：采用 C++编程语言开发的系统后端，负责业务逻辑处理、接口开发、数据交互等功能；
- MySQL 数据库：一款免费开源的关系型数据库管理系统，用于数据存储、查询与管理；
- ER 图：实体-关系图，用于描述数据库中实体之间的关系，辅助数据库设计；

- CRUD：Create（新增）、Read（查询）、Update（更新）、Delete（删除），是数据库操作的核心功能；
- 接口联调：前端与后端通过接口进行数据交互，排查数据不一致、接口异常等问题，确保系统功能正常；
- 模块化设计：将系统拆分为多个独立的模块，每个模块负责特定的功能，便于开发、维护与扩展。

11.3 缩略语表（按字母顺序排列）

- CASE：Computer-Aided Software Engineering（计算机辅助软件工程）；
- CSS：Cascading Style Sheets（层叠样式表）；
- HTML：HyperText Markup Language（超文本标记语言）；
- JS：JavaScript（一种脚本语言，用于前端交互）；
- MySQL：My Structured Query Language（关系型数据库管理系统）；
- UI：User Interface（用户界面）。

12 软件开发过程模型与敏捷开发对比分析

12.1 传统软件开发过程模型与敏捷开发对比

对比维度	传统软件开发模型（瀑布模型）	敏捷开发（Scrum）
开发流程	线性顺序，需求→设计→编码→测试→交付，阶段边界清晰	迭代增量，分 Sprint 周期，持续迭代、快速反馈

需求适应性	需求固定，变更成本高、难度大	需求灵活，可在迭代中调整，适配变化
文档要求	文档驱动，需编写大量完整前期文档	代码驱动，文档精简，优先可用软件与交付
团队协作	分工明确、层级清晰，沟通较少	跨职能紧密协作，每日站会，高频沟通
交付方式	项目末期一次性交付	迭代周期内持续交付可用功能模块
风险控制	后期发现问题，修复成本高	早期暴露问题，逐轮降低风险

12.2 优缺点分析

- **瀑布模型优点**
  - ✓ 流程规范，阶段清晰，便于把控进度与质量。
  - ✓ 文档完整，便于后期维护与交接。
  - ✓ 适合需求稳定、规模较小的项目。
- **瀑布模型缺点**
  - ✓ 需求无法变更，难以适配实际开发调整。
  - ✓ 测试滞后，问题发现晚，修复成本高。
  - ✓ 用户参与度低，交付成果可能偏离预期。
- **敏捷开发（Scrum）优点**

- ✓ 迭代开发，快速响应需求变化。
- ✓ 持续交付，阶段性可验证成果。
- ✓ 团队高频沟通，问题及时解决。
- ✓ 用户参与度高，产品更贴合使用需求。

❀ **敏捷开发（Scrum）缺点**

- ✓ 对团队自律与协作能力要求高。
- ✓ 文档较少，不利于长期维护。
- ✓ 迭代节奏快，易出现进度压力。

12.3 在本博客项目中的应用选择

本项目采用瀑布模型 + Scrum 敏捷混合模式：

1. 前期需求分析、可行性分析、架构设计使用瀑布模型，保证基础规范。
2. 开发、测试、联调采用 Scrum 敏捷，按 1 周 1 个 Sprint 迭代，共 5 个迭代周期。
3. 每周完成核心功能交付，及时测试与修复，适配小范围需求调整。

13 Scrum 开发方法应用

13.1 Scrum 角色与职责

1. Product Owner（产品负责人）：确定需求优先级，审核功能完成度。

2. Scrum Master（**敏捷主管**）：保障流程顺畅，解决团队障碍。

3. Development Team（**开发团队**）：5 人小组，跨职能完成设计、开发、测试、文档。

13.2 Scrum 核心流程

1. **产品待办列表（Product Backlog）**
    - 用户注册登录、文章管理、评论互动、管理员后台、界面优化。
2. **Sprint 计划会议**
    - 周期：1 周 / Sprint，共 5 周。
    - 每轮确定本轮开发任务与目标。
3. **每日站会**
    - 10 分钟，同步进度、暴露问题。
4. **Sprint 评审会议**
    - 演示本轮完成功能，收集改进意见。
5. **Sprint 回顾会议**
    - 总结流程问题，优化下一轮开发。

13.3 项目 Sprint 规划（5 周）

- Sprint 1：需求确认、环境搭建、数据库设计。
- Sprint 2：用户注册登录模块开发、接口开发。
- Sprint 3：文章发布 / 编辑 / 删除、评论功能开发。
- Sprint 4：管理员后台开发、前后端联调。
- Sprint 5：系统测试、bug 修复、文档完善、交付。

14 项目风险分级与应对预案

14.1 风险分级标准

- **高风险**：影响项目成败、导致延期 / 失败。
- **中风险**：影响功能质量与进度，可补救。
- **低风险**：局部问题，不影响整体交付。

14.2 风险清单与应对预案

风险编号	风险描述	风险等级	应对预案
R1	C++ 后端 HTTP 服务开发困难	高	提前学习 cpp-httplib 库，参考开源案例，分工攻坚
R2	前后端接口联调失败	中	提前约定接口文档，分模块联调，每日验证
R3	数据库设计不合理	中	先画 E-R 图，小组评审通过再生成 SQL 脚本
R4	团队进度不一致、延期	中	使用 PingCode 跟踪任务，每周固定进度会议
R5	工具兼容 / 环境问题	低	统一开发环境与版本，提前搭建测试
R6	安全漏洞	中	密码加密存储，接口权限控制
R7	人员时间冲突	中	提前排期，任务备份，成员可互相支援

R8	测试不充分、遗留 Bug	中	全员参与测试，用 JMeter 做接口自动化测试

14.3 风险管理流程

1. 风险识别：每周会议识别新风险。
2. 风险评估：分级并记录。
3. 风险应对：制定预案并执行。
4. 风险监控：跟踪状态，闭环管理。

附录

本附录按字母顺序编排，提供便于文档维护、补充说明的相关信息：

附录 A：团队分工表

分工角色	人数	核心职责
前端开发	1 人	负责页面开发、交互实现，对接后端接口，适配 PC 端界面，配合美工优化视觉效果。
C++后端开发	2 人	分工负责接口开发、业务逻辑处理，对接 MySQL 数据库，使用 Apache JMeter 进行接口测试，互相审核代码。
数据库设计	1 人	负责数据库表结构设计、ER 图绘制（使用 Microsoft Visio），生成 SQL 脚本，配合后端对接数据库。

美工+文档协调	1 人	负责界面美化、素材设计，编写项目文档，使用 PingCode 制定甘特图、跟踪开发进度，协调团队沟通。

附录 B：数据库表结构设计（SQL 脚本）（待完成）

附录 C：技术栈相关教程链接

- MySQL 8.0 官方手册：MySQL :: MySQL Documentation
- MySQL Connector/C++开发文档：MySQL :: MySQL Connector/C++ Developer Guide
- MySQL Workbench 使用教程：MySQL :: MySQL Workbench Manual
- VS Code C++开发教程：C/C++ for Visual Studio Code
- Github 官方使用手册：Hello World - GitHub Docs